

RUN & USER GUIDE

File Net

A local-first file sharing platform — React, Supabase & PowerSync

Version 2 — full rebuild from the original Flutter/Firebase app

Prepared: July 17, 2026

Covers: local development setup, day-to-day usage, admin operations, and the fully offline self-hosted deployment

Table of Contents

- 1. What This Is
- 2. Running It Locally (Development)
- 3. Default Admin Login
- 4. Using the System — Everyone
- 5. Using the System — Admin Features
- 6. File Versioning Explained
- 7. Offline & Sync Behavior
- 8. Fully Offline / Self-Hosted Deployment
- 9. Troubleshooting
- 10. Reference — Ports, URLs & Files

1. What This Is

File Net is a file-sharing platform for an organization: people upload documents, share them with individual colleagues or entire groups, and get notified when something is shared with them. This is a full rebuild of the original Flutter + Firebase app on a new stack, built specifically to support:

- **Group sharing** — admins create a group; sharing to that group delivers the file to every member.
- **Notifications** — every share generates an in-app notification for the recipient.
- **Offline-capable database** — the app works online and offline from the same codebase, syncing automatically when connectivity returns.
- **A real user dashboard** — everyone gets a home screen showing what's been shared with them and quick access to share their own files.
- **File versioning** — a file can be re-uploaded as a new version without losing history; every past version stays downloadable.

Technology at a glance

Layer	Technology
Frontend	React + Vite + TypeScript, Tailwind CSS + shadcn/ui
Local-first sync	PowerSync — an embedded local database (SQLite, in the browser) that syncs with Postgres, so the app works instantly and offline
Backend	Supabase — Postgres, Auth, Storage. No Edge Functions: every business rule (group fan-out, notifications, account provisioning) lives directly in Postgres
Offline packaging	Installable PWA (works without internet once loaded) + a self-hosted Docker Compose bundle for a fully offline, on-prem deployment

2. Running It Locally (Development)

Three things need to be running together. Each is started with one command, in this order:

2.1 Prerequisites

- Node.js 22+ and npm
- Docker Desktop (running)
- Supabase CLI (`npm install -g supabase` or via Homebrew)

2.2 Start the backend

```
cd filenet-web
npx supabase start      # Postgres, Auth, Storage – local Supabase stack
npm run dev:powersync   # starts the PowerSync sync service (Docker)
npm run dev:ui          # starts the app itself
```

Open **`http://localhost:5173`** in your browser.

First time only: a fresh `supabase start` gives you an empty database — no accounts exist yet. Either register a normal account at `/register` and promote it to admin (see below), or use the default admin credentials already seeded in this environment (Section 3).

2.3 Creating the very first admin (fresh database)

Granting admin access normally requires being an admin already — a chicken-and-egg problem for a brand new database. A one-time bootstrap function solves this: it works exactly once, only while zero admins exist in the system.

1. Register a normal account at `/register` .

2. Run this once against the database (Supabase Studio's SQL Editor, or `psql`):

```
select bootstrap_first_admin('the-email-you-registered@example.com');
```

3. Log out and back in — the account now has the admin role.

A second call to this function is deliberately rejected once any admin exists, so it can't be used to silently self-promote later.

2.4 Resetting the database

```
npx supabase db reset
```

This wipes all data and re-applies the schema from scratch — useful during development, but it also removes every account including admin. Re-run the bootstrap step above afterward.

3. Default Admin Login

This environment already has an admin account seeded and ready to use:

EMAIL

admin@filenet.local

PASSWORD

TestPass123!

Change this before any real deployment. This is a development-environment convenience account. For the self-hosted/offline deployment (Section 8), you create a fresh admin yourself via the bootstrap function — there is no default password baked into the production path.

After logging in as admin, the sidebar shows an **Admin** section with three extra pages: **Overview** (platform stats), **Groups**, and **Users** — covered in Section 5.

4. Using the System — Everyone

4.1 Creating an account

Go to `/register`, enter your name, email, phone (optional), and a password. You're taken to the login screen — sign in with the same credentials.

4.2 The Dashboard

Your home screen after logging in. Shows how many files you own, how many people are on File Net, and a live sync status indicator (*Online — up to date* or *Offline — showing cached data*).

4.3 My Files

Everything you've uploaded. Click **New File**, give it a title and description, choose a PDF (25MB limit), and upload. Click any file card to open its detail page.

4.4 Sharing a file

From a file's detail page, click **Share**. Two tabs:

- **Person** — search by name or email, select someone. They get the file and a notification instantly (once online).
- **Group** — pick a group you belong to. Every member of that group receives the file and a notification in one action.

Only admins can *create* groups (Section 5.1) — but any file owner can share into a group they're a member of.

4.5 Shared With Me

Files other people have shared with you, either directly or through a group. Click through to view version history and download.

4.6 Notifications

A bell icon in the sidebar shows an unread count. The Notifications page lists every share you've received — click one to jump straight to that file. Opening a notification marks it read.

4.7 Account & password

The Account page shows your profile and lets you change your password. If your account was admin-provisioned (Section 5.3), you'll be forced here on first login until you set your own password.

5. Using the System — Admin Features

5.1 Groups

`/groups` — create a group (name + description), then **Manage members** to add or remove people. Anyone in the group can be shared files by any file owner, not just the admin who created it.

5.2 Users

`/admin/users` — every account on the platform, with role and status. **Block / Unblock** toggles access instantly: a blocked user is signed out and redirected to a "blocked" page the moment their session next syncs (including on their very next login attempt).

5.3 Provisioning an account directly

Instead of waiting for someone to self-register, click **Provision account**, fill in their name, email, phone, and role (office / lecturer / admin). A random password is generated and shown once — copy it and hand it to that person. They're required to set their own password the first time they log in.

5.4 Admin Overview

`/admin/dashboard` — live counts: total files, total users, total groups, and how many users are currently blocked.

6. File Versioning Explained

A file in File Net is really two things: a **document identity** (title, description, owner — this never changes) and a list of **versions** (the actual uploaded PDFs, in order).

Upload a file → that's version 1.

Upload new version from the file's detail page → adds version 2, keeping version 1 intact.

The version list shows every version with who uploaded it, when, and file size — the newest is labeled **(latest)**.

Anyone the file is shared with sees the *entire* version history, not just the latest — useful for tracking a document's evolution.

Only the file's owner (or an admin) can add a new version. People it's shared with can view and download every version, but can't overwrite them.

Saving a version for offline viewing

Each version row has a small download-to-device icon — click it to store that PDF locally so it opens even without an internet connection later. A second click removes it from local storage.

7. Offline & Sync Behavior

File Net is **local-first**: the app reads and writes to a database embedded in your browser first, then syncs with the server in the background. This means:

- The app loads instantly and keeps working if your connection drops — you'll see recently-synced files, shares, and notifications exactly as before.
- Actions you take offline (creating a file record, editing) are queued locally and sent to the server automatically once you're back online.
- **Uploading or downloading the actual PDF file itself still requires a connection** — only the metadata (titles, shares, notifications) works fully offline. Use the "save for offline" button (Section 6) to pre-download specific files you know you'll need without a connection.
- Installing the app (browser's "Install App" / "Add to Home Screen" prompt) makes it launch like a native app and improves offline reliability further.

The Dashboard's sync status indicator always tells you which mode you're currently in.

8. Fully Offline / Self-Hosted Deployment

For a site with no reliable internet at all, the entire backend can run on one machine on the local network — no cloud dependency whatsoever. This lives in `docker/self-hosted/` in the project.

1. `cd docker/self-hosted && cp .env.example .env` — fill in real secrets (the file has generation instructions for each one).
2. `docker compose up -d --build` — brings up Postgres, Auth, Storage, PowerSync, and the app itself, all in containers.
3. Apply the schema (same migration files used everywhere else) directly to the containerized Postgres.
4. Register an account through the running app, then bootstrap it as admin exactly as in Section 2.3.
5. Set up two Vault secrets (documented in the bundle's README) so admin account provisioning works.

Service	Default port
The app	80
Supabase API (auth/rest/storage)	8000
PowerSync	8080
Database admin UI (optional)	8082

Deliberately excluded from this deployment: **Edge Functions** and **Realtime** — nothing in File Net uses either (all server-side logic is plain Postgres functions/triggers; PowerSync handles live sync on its own connection). Full details and exact commands are in `docker/self-hosted/README.md`.

This deployment path has been checked for correct configuration and service wiring, but has not yet been run through a full live test the way the day-to-day app has. Walk through registration → admin bootstrap → upload → share once on the real hardware before relying on it for production use.

9. Troubleshooting

Symptom	Likely cause / fix
App loads but nothing syncs, status stuck on "connecting"	PowerSync container isn't running — check <code>docker ps</code> for <code>powersync_demo</code> ; restart with <code>npm run dev:powersync</code> .
"Port 5173 already in use" or the app looks wrong / unbranded	Another project's dev server may be occupying the port. Check <code>lsof -i :5173</code> and confirm the title bar says "File Net" before assuming the app is broken.
Can't log in as the very first admin	Run <code>bootstrap_first_admin('email')</code> once against the database — see Section 2.3. It only works while zero admins exist.
Provisioning an account fails with a Vault-related error	The two Vault secrets (<code>service_role_key</code> , <code>gotrue_admin_url</code>) haven't been set up in this database yet — see the setup steps in Section 2 / 8.
Sharing to a group doesn't seem to do anything	Make sure the group actually has members (Admin → Groups → Manage members) — sharing to an empty group correctly delivers to no one.
A blocked user can still see old data	Enforcement applies on next sync/login, not mid-session instantly — have them refresh or sign out and back in.

10. Reference — Ports, URLs & Files

Local development

What	URL
The app	<code>http://localhost:5173</code>
Supabase API	<code>http://127.0.0.1:54321</code>
Supabase Studio (browse tables, run SQL)	<code>http://127.0.0.1:54323</code>
PowerSync sync endpoint	<code>http://localhost:8080</code>

Key project locations

Path	What's there
<code>supabase/migrations/</code>	The entire database schema, RLS policies, and business-logic functions
<code>docker/powersync.yaml</code>	Sync rules — what data each user's device is allowed to download
<code>docker/self-hosted/</code>	The fully offline deployment bundle
<code>src/features/</code>	App feature code (auth, files, groups, admin, notifications)
<code>src/app/router.tsx</code>	Every route and which role can access it